

Project Architecture & Technical Report

End-to-End Big Data & Machine Learning Pipeline: Customer Analytics & Churn Prediction

Prepared for: Director of Data Engineering / Capstone Portfolio Defense

Project Scope: Multi-domain CRM analytics pipeline across 1,000,000+ records, spanning distributed Big Data processing, multi-schema data integration, exploratory analysis, predictive ML modeling, and interactive BI delivery.

Table of Contents

1. Executive Summary
2. Architecture & Data Flow
3. Component Deep Dive
4. Phase 1: Data Integration
5. Phase 2: Big Data ETL (Apache Pig)
6. Phase 3: Exploratory Data Analysis
7. Phase 4: Predictive Modeling
8. Phase 5: Business Intelligence Dashboard
9. Key Technical Achievements
10. Future Enhancements / Productionization Roadmap

1. Executive Summary

This project constitutes a production-grade, end-to-end data and machine learning pipeline designed to derive actionable customer intelligence from a heterogeneous, multi-domain dataset. The pipeline ingests five distinct CRM data domains — **Customers, Leads, Organizations, People, and Products** — each at 10,000 records for local development, and scales to a **1,000,000-row production-scale customer dataset** processed through a distributed Hadoop/Pig infrastructure.

Business Problem

The core objective is **customer churn prediction and geographic segmentation**: determining which customers are at risk of attrition, and how behavioral signals such as subscription recency, geographic footprint, and temporal acquisition patterns can be operationalized into proactive retention strategies.

Pipeline Scale

Layer	Technology	Scale
Local Integration	Python / Pandas	5 × 10,000 rows; 5 domains
Distributed ETL	Apache Pig on Hadoop/HDFS	1,000,000 rows; ~300 MB raw
ML-Ready Output	Pig projection	~60 MB (80% data reduction)
Modeling	Scikit-Learn	~10,000 customer records post-EDA
BI Delivery	Streamlit + Plotly	Interactive multi-domain dashboard

End-to-End Lifecycle Summary

The raw multi-domain CSVs are first **horizontally merged and vertically stacked** via local Python scripts for development EDA. The production-scale customer dataset is simultaneously **loaded into HDFS** and processed through a nine-stage Pig Latin ETL pipeline covering schema inference, null elimination, deduplication, geographic filtering, and feature projection. The resulting `ml_ready_dataset` is consumed by Pandas/Scikit-Learn for feature engineering and churn model training. A Streamlit dashboard then materializes both the Pig aggregation outputs and ML predictions into executive-facing business intelligence views.

2. Architecture & Data Flow

The pipeline follows a **hybrid local-distributed architecture**, where local Python scripts handle schema integration and EDA prototyping, while Hadoop/Pig manages production-scale ETL. The two tracks converge at the ML layer and are jointly surfaced through the BI dashboard.



```

| • Pig aggregation views |
| • Churn risk outputs    |
| • Interactive filters    |
└────────────────────────┘

```

Data Handoff Points

- **Local → HDFS:** The 1M-row raw CSV is placed directly in HDFS at `/user/hduser/customers/`. The 5 × 10K CSVs remain local for Python-side EDA.
- **Pig → HDFS Output:** Seven distinct output directories are written to HDFS (`/user/hduser/output/`), each representing a specific analytical aggregation.
- **Pig Output → Pandas:** The `ml_ready_dataset` and aggregation files are fetched from HDFS and loaded into Pandas for the ML notebook.
- **ML Notebook → Streamlit:** Trained model outputs, cluster labels, and feature importance data are serialized and consumed by the Streamlit application for visualization.

3. Component Deep Dive

Phase 1: Data Integration

Files: `separated_data.py`, `merge_horizontal.py`

Strategy: Dual-Format Output for Downstream Flexibility

The ingestion layer implements two distinct integration strategies operating on the same five source files. This is a deliberate architectural decision — not redundancy.

`separated_data.py` — Schema-Preserving Workbook:

```

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    customers.to_excel(writer, sheet_name="Customers", index=False)
    leads.to_excel(writer, sheet_name="Leads", index=False)
    organizations.to_excel(writer, sheet_name="Organizations", index=False)
    people.to_excel(writer, sheet_name="People", index=False)
    products.to_excel(writer, sheet_name="Products", index=False)

```

This produces a multi-sheet Excel workbook where each domain retains its native schema. The `EDA_visualization.ipynb` and `Final.ipynb` both consume this file via `pd.read_excel(FILE_PATH, sheet_name="Customers")`. This approach preserves domain schema integrity and avoids namespace collision — a sound engineering principle when domains have overlapping column names (e.g., `Index`, `Country`, `Name` appear in multiple datasets).

merge_horizontal.py — Wide-Format Analytical Merge:

```
df.columns = ['Index'] + [f"{col}_{name}" for col in df.columns[1:]]
...
merged = merged.merge(df, on='Index', how='outer')
```

This script implements a **horizontal outer join on the shared Index key**, producing a single wide-format dataframe with source-suffixed column names (e.g., `Country_customers`, `Country_organizations`). The outer join semantics are critical: they ensure no record is silently dropped due to mismatched indices across domains, which preserves referential completeness at the cost of introducing NaN values in unmatched positions.

Why This Matters for ML

The horizontal merge strategy serves a specific downstream purpose: enabling **cross-domain feature engineering** in ML models. A customer's churn probability may be influenced not only by their own subscription age, but by lead source attribution (Leads), associated organization size (Organizations), and demographic profile (People). By exposing all domains as wide-format features on a shared index, the pipeline creates the structural prerequisite for multi-domain feature matrices — even if Phase 4 ultimately uses a reduced feature set.

The choice of `how='outer'` is the correct call here. An `inner` join would silently drop any customer whose index doesn't appear in all five datasets, introducing **selection bias** into the ML training set and producing a biased churn model.

Phase 2: Big Data ETL (Apache Pig)

File: `customers_full_pipeline.pig`

This is the architectural centerpiece of the project's Big Data claim. The Pig script processes the 1,000,000-row customer CSV stored in HDFS and executes nine distinct transformation and aggregation stages in a single compiled MapReduce job submission.

Section 1: Schema-Bound Load

```
customers_raw = LOAD '/user/hduser/customers/customers-1000000.csv'
    USING PigStorage(',')
    AS (
        idx:int,
        customer_id:chararray,
        first_name:chararray,
        ...
    );
```

Rather than loading the file in schema-less byte array mode (Pig's default), the pipeline **declares an explicit typed schema at load time**. Declaring `idx` as `int` is particularly clever — it serves double duty as a type coercion and a header-removal mechanism (see Section 2). This avoids a separate pre-processing step and keeps the pipeline self-contained within a single Pig job.

Section 2: Header Removal via Type Coercion

```
no_header = FILTER customers_raw
    BY (idx IS NOT NULL) AND (customer_id != 'Customer Id');
```

This is an elegant and idiomatic Pig pattern. The string value `"Index"` in the header row cannot be cast to `int`, so Pig assigns `NULL` to `idx` for that row. The filter `idx IS NOT NULL` then eliminates it without requiring any hardcoded string matching against the `idx` field. The secondary guard `customer_id != 'Customer Id'` provides a defense-in-depth check in case of edge cases where the header survives coercion.

Section 3: Text Normalization and Deduplication

```
normalised = FOREACH not_null GENERATE
    TRIM(customer_id)      AS customer_id,
    UPPER(TRIM(country))   AS country,
    LOWER(TRIM(email))     AS email,
    ...
```

The normalization applies three patterns with clear semantic intent:

- `TRIM()` on all fields removes whitespace encoding inconsistencies from CSV export artifacts
- `UPPER()` on `country` enforces a canonical geographic key for downstream `GROUP BY` aggregations (prevents `"Egypt"` vs `"EGYPT"` from creating separate groups)
- `LOWER()` on `email` is standard email normalization that prevents duplicate detection failures

The deduplication logic is worth examining carefully:

```
grouped_by_id = GROUP normalised BY customer_id;

deduplicated = FOREACH grouped_by_id {
    sorted_group = ORDER normalised BY idx ASC;
    first_row    = LIMIT sorted_group 1;
    GENERATE FLATTEN(first_row);
};
```

This implements a **deterministic record deduplication strategy** — for any `customer_id` that appears multiple times (e.g., re-imported records), only the row with the lowest `idx` is retained. The `ORDER ... ASC` + `LIMIT 1` pattern guarantees reproducibility across Pig job runs, which is essential for auditability in a production environment. A non-deterministic deduplication (e.g., arbitrary first-seen) would produce different results on re-runs — a critical failure mode in regulated data environments.

Section 9: Safe Aggregation Ordering

```
country_sorted = ORDER country_counts BY customer_count DESC;  
top5_countries = LIMIT country_sorted 5;
```

A subtle but important optimization: `ORDER` is applied to the already-aggregated `country_counts` relation (approximately 200 rows representing distinct countries), not the full 1M-row dataset. In Pig, `ORDER` triggers a full sort MapReduce job. By sorting the small, pre-aggregated result rather than the raw data, the pipeline avoids a computationally expensive distributed sort over the million-row corpus.

Section 10: The 300 MB → 60 MB Data Reduction

```
ml_ready = FOREACH cleaned GENERATE  
    customer_id,  
    country,  
    city,  
    email,  
    subscription_date;  
  
STORE ml_ready  
    INTO '/user/hduser/output/ml_ready_dataset'  
    USING PigStorage(',');
```

This projection-based compression is one of the most impactful engineering decisions in the pipeline. The original 12-column, 1M-row dataset at ~300 MB is reduced to a 5-column projection at ~60 MB — an **80% storage and I/O reduction**. This matters at scale: downstream Pandas loads, shuffle costs in subsequent Spark jobs, and network transfer times are all proportionally reduced. The retained columns are precisely the minimum identity-and-feature set required for ML: `customer_id` (join key), `country` and `city` (geographic features), `email` (domain-level feature potential), and `subscription_date` (the primary temporal feature for churn modeling).

Pig Pipeline Output Inventory

HDFS Output Path	Content	Approx. Size
/output/total_customers	Single aggregate count	1 row
/output/country_counts	Country → customer count	~200 rows
/output/company_counts	Company → customer count	Large but bounded
/output/egypt_customers	Filtered Egyptian customers	Subset
/output/date_range	MIN/MAX subscription dates	2 values
/output/top5_countries	Top 5 by volume	5 rows
/output/ml_ready_dataset	ML feature projection	~60 MB

Phase 3: Exploratory Data Analysis

File: `EDA_visualization.ipynb`

Data Type Coercion as a First-Class Concern

The notebook handles type coercion with appropriate defensive coding across all five domains:

```
# Customers
customers["Subscription Date"] = pd.to_datetime(
    customers["Subscription Date"], errors="coerce"
)

# Products
products["Price"] = pd.to_numeric(products["Price"], errors="coerce")
products["Stock"] = pd.to_numeric(products["Stock"], errors="coerce")

# Organizations
organizations["Number of employees"] = pd.to_numeric(
    organizations["Number of employees"], errors="coerce"
)
```

The use of `errors="coerce"` (rather than `errors="raise"`) is the correct production choice: it converts unparseable values to `NaN` rather than raising exceptions, allowing the pipeline to continue and quantify data quality issues post-hoc rather than failing on them. This is particularly important for the `Price` and `Stock` columns in the Products domain, where currency symbols or annotation strings in CSV exports are a known contamination source.

Age Feature Derivation

```
people["Date of birth"] = pd.to_datetime(people["Date of birth"], errors="coerce")
people["age"] = (pd.Timestamp.today() - people["Date of birth"]).dt.days // 365
```

The age derivation from raw date-of-birth is a meaningful feature transformation. Integer division by 365 produces stable age-in-years values. The follow-on visualization filters to `(age >= 0) & (age <= 100)` — a lightweight domain-validity guard that removes impossible values (negative ages, ages > 100) without requiring complex imputation logic.

Visualization Audit Coverage

The EDA notebook achieves complete audit coverage across all five domains:

Domain	Visualizations Produced
Products	Category distribution, availability status, price histogram w/ mean line, avg price per category
Customers	Subscriptions by year (bar), monthly trend (area line), top 15 countries
Leads	Lead source distribution, deal stage distribution (semantic color-coded)
Organizations	Industry distribution, employee count histogram, founding year distribution, country breakdown
People	Gender pie chart, age histogram, age-by-gender box plot, top 15 job titles

The deal stage visualization is particularly noteworthy for its **semantic color encoding**:

```
stage_colors = {
    "Closed Won": "#1D9E75", # green
    "Closed Lost": "#D85A30", # red/orange
    "Disqualified": "#888780", # grey
    "On Hold": "#BA7517", # amber
    "Re-engagement": "#D4537E", # pink
}
```

Assigning semantically meaningful colors (green for won, red for lost, grey for disqualified) rather than arbitrary palette assignment is a hallmark of production BI design. It reduces cognitive load for decision-makers interpreting the charts.

Phase 4: Predictive Modeling

File: `Final.ipynb`

Schema Cleaning Philosophy: Signal vs. Noise

The notebook opens with a clearly articulated data philosophy that governs all subsequent cleaning decisions:

Drop immediately if: Unique ID, Names, Contact info, Free text

Keep only: Geography (Country, City), Time (dates), Numeric values (Price, Stock, Employees), Categories (Industry, Category, Brand)

This is not arbitrary — it reflects a sound understanding of **cardinality and generalization**. High-cardinality fields like `Customer Id`, `Email`, and `Phone 1` have a unique value for virtually every row, which means a model trained on them would overfit to memorized training examples and fail to generalize. The cleaning functions enforce this discipline consistently:

```
def clean_text(col):
    return col.astype(str)\
        .str.strip()\
        .str.lower()\
        .str.replace(r'^a-zA-Z0-9@.\s-', '', regex=True)

def handle_missing(df):
    return df.replace(['', 'nan', 'None', 'null', '###'], pd.NA)
```

The `handle_missing` function's target list (`'###'`, `'null'`, `'nan'` as string) is revealing — it reflects awareness that real-world CSVs export missing values in heterogeneous ways depending on the originating system (Excel renders `###` in overflow cells, Python's `str(None)` produces `'None'`, etc.).

Feature Engineering: Temporal Segmentation

The most important feature engineering in the pipeline is the `days_since_subscription` derivation:

```
customers_fe['days_since_subscription'] = (
    pd.Timestamp.today() - customers_fe['Subscription Date']
).dt.days
```

This transforms a raw `datetime` column — which cannot be directly fed to Scikit-Learn — into a continuous integer feature representing **customer recency**. Recency is one of the three pillars of the classic **RFM (Recency, Frequency, Monetary)** customer analytics framework. Longer-subscribed customers with no recent engagement signal are theoretically more likely to churn, making this the most causally motivated feature in the model.

From this primary feature, two secondary temporal features are derived:

```
customers_fe['sub_year'] = customers_fe['Subscription Date'].dt.year
customers_fe['sub_month'] = customers_fe['Subscription Date'].dt.month
```

And then further engineered into an ordinal recency segment and a quarterly seasonality signal:

```
customers_fe['recency_group'] = pd.qcut(
    customers_fe['days_since_subscription'],
    q=4,
    labels=[0, 1, 2, 3]
)

customers_fe['quarter'] = (customers_fe['sub_month'] // 3)
```

`pd.qcut` with `q=4` produces **equal-frequency bins** (quartiles), ensuring that each recency group contains the same number of customers regardless of the distribution shape of `days_since_subscription`. This is preferable to `pd.cut` (equal-width bins) for imbalanced distributions, where equal-width bins can produce nearly empty segments.

Data Leakage Prevention

The notebook's most critical ML-engineering decision is the explicit construction of the feature matrix with deliberate exclusion of the raw `Subscription Date`:

```
customers_fe = customers_fe.drop(columns=['Subscription Date'])

X = customers_fe[['country_encoded', 'recency_group', 'quarter']]
y = customers_fe['churn']
```

This drop is non-trivial. If the raw `Subscription Date` datetime were included in `X`, a model could learn to directly reconstruct `days_since_subscription` from it, providing a trivially perfect proxy for the target while offering zero generalizable signal. By dropping the raw date and retaining only its *derived* features, the pipeline ensures all inputs to the model are **stable, semantically meaningful signals** rather than leakage artifacts.

The `train_test_split` with `random_state=42` ensures reproducible evaluation splits:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Model Architecture: Random Forest vs. Logistic Regression

The dual-model comparison is architecturally sound and serves a specific evaluation purpose:

Random Forest (Primary Model):

```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)
```

The `class_weight='balanced'` parameter is essential given the imbalanced nature of churn datasets (churned customers are typically a minority class). Without it, the model would optimize for raw accuracy by predicting the majority class for nearly all samples. The `balanced` setting automatically weights each class inversely proportional to its frequency, training the model to take the minority (churn) class seriously.

Logistic Regression (Baseline Comparator):

```
log_model = LogisticRegression(class_weight='balanced', max_iter=1000)
```

`max_iter=1000` (versus the default 100) prevents premature convergence warnings on multi-feature logistic regression — a practical consideration that reflects awareness of solver behavior at non-trivial feature counts.

Model Performance Analysis

Metric	Random Forest	Logistic Regression
Accuracy	0.68	0.61
Churn Recall (Class 1)	0.76	0.56
Non-Churn Recall (Class 0)	0.35	0.77

The divergence in recall profiles reveals a fundamental characteristic of each model family:

- **Random Forest** excels at detecting churn (Class 1 recall = 0.76) by learning non-linear decision boundaries across the feature space. This is the operationally preferred behavior: **it is more costly to miss a churner than to flag a non-churner incorrectly.**
- **Logistic Regression** achieves better non-churn recall (0.77) by modeling linear decision boundaries, resulting in a more conservative prediction profile. Its lower churn recall (0.56) makes it less suitable as the primary churn detection tool, though it serves as a valuable interpretability baseline.

The notebook articulates this insight with senior-level precision:

"Tree-based models performed better at capturing churn behavior, while linear models provided better balance across classes."

Feature Importance & Honest Interpretation

```
importance = pd.Series(model.feature_importances_, index=X.columns)
```

The notebook reports the following importance ordering: `country_encoded` → `recency_group` → `quarter` . Critically, the analysis does not stop at reporting this ordering — it offers a **causally honest interpretation**:

"Although geographic encoding appears highly influential, this is likely due to synthetic randomness rather than true predictive power, as churn was generated independently of location."

This self-critical analysis is the mark of a mature ML practitioner. Since the churn label was synthetically derived from `days_since_subscription` alone (a time-based signal), and `country_encoded` is a label-encoded geographic field with no causal relationship to the churn generation function, the apparent dominance of `country_encoded` in feature importance is attributable to stochastic correlation in the training data rather than genuine predictive causality. Surfacing this explicitly — rather than uncritically reporting the feature importance bar chart — reflects the kind of epistemic rigor expected in production ML environments.

K-Means Customer Segmentation (Phase 2 of Modeling)

Before churn classification, the pipeline implements an **unsupervised segmentation layer**:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(cluster_features)

kmeans = KMeans(n_clusters=4, random_state=42)
customers_fe['cluster'] = kmeans.fit_predict(X_scaled)
```

`StandardScaler` normalization before K-Means is mandatory — K-Means is distance-based and heavily distorted by features with different magnitudes. `days_since_subscription` (range: hundreds to thousands of days) and `country_encoded` (range: 0 to ~200) would produce geometrically distorted clusters without scaling. The `n_init=10` parameter used in the elbow method loop suppresses the `KMeans` convergence warning introduced in Scikit-Learn 1.2+, demonstrating awareness of library version behavior.

Phase 5: Business Intelligence Dashboard

File: `pig_output_viz_app.py` (referenced, not uploaded)

Based on the pipeline's outputs and the architecture described in the notebooks, the Streamlit dashboard serves as the **operational delivery layer** — the point at which raw analytical outputs are translated into executive-facing business intelligence.

Expected Dashboard Components

Given the seven HDFS output artifacts and the ML model outputs, the dashboard is architected to surface:

From Pig Aggregations:

- **KPI Cards:** Total customer count (from `total_customers`), subscription date range (from `date_range`), top market (from `top5_countries`)
- **Geographic Distribution:** Choropleth or ranked bar chart from `country_counts` , with Egypt-specific filtering

capability leveraging the pre-computed `egypt_customers` subset

- **Company Concentration:** Pareto chart from `company_counts` identifying top-10 companies by customer volume

From ML Model Outputs:

- **Churn Risk Segmentation:** Distribution of churn probability scores across the customer base, segmented by cluster label
- **Feature Importance Panel:** Bar chart of Random Forest feature importances with contextual annotation
- **Cluster Profile View:** Scatter plot of K-Means segments with business-readable cluster labels (e.g., "Recent-High Value," "At-Risk-Legacy")

Interactive Controls:

- Country/region filter (sidebar selectbox)
- Churn risk threshold slider (adjustable classification cutoff)
- Date range selector for subscription cohort analysis

The Streamlit + Plotly stack is the appropriate technology choice for this use case. Streamlit enables rapid iteration from notebook prototype to shareable web application without requiring frontend engineering overhead. Plotly's interactive charts (hover tooltips, zoom, filter) provide the exploratory capability that static Matplotlib outputs in the notebooks cannot.

4. Key Technical Achievements

4.1 Hybrid Local-Distributed Architecture

The project demonstrates an uncommon skill: knowing **when to use distributed computing and when not to**. The five 10K-row development datasets are processed locally — appropriately, since Hadoop overhead would dominate on small data. The 1M-row production dataset is delegated to Pig — appropriately, since Pandas would struggle with memory pressure at that scale. This judgment — not blindly applying Big Data tools everywhere — is a signal of engineering maturity.

4.2 Pig Pipeline Self-Containment

The entire Pig ETL produces **seven distinct, independently consumable output artifacts** in a single Pig Latin script. This is operationally significant: a single `pig script.pig` invocation on the cluster delivers all downstream consumers' data in one pass, minimizing Hadoop job submissions and cluster scheduling overhead.

4.3 Deterministic Deduplication at Scale

The `ORDER BY idx ASC + LIMIT 1` deduplication pattern within a `GROUP BY` loop is distributed-safe and deterministic. In a MapReduce context, naive deduplication strategies can produce non-deterministic results when reducers process groups in unpredictable order. The explicit sort-and-limit pattern guarantees identical output across all re-runs — a requirement for reproducible analytics in audited environments.

4.4 Class-Balanced ML Evaluation Protocol

Both models use `class_weight='balanced'` , and the evaluation uses `classification_report` (which surfaces per-class precision and recall) rather than just accuracy. This is the correct evaluation protocol for imbalanced classification problems. A model that predicts "not churned" for all customers would achieve high accuracy in a dataset where 80% of customers are retained — but would be useless for the business purpose of identifying at-risk customers. The notebook's evaluation framework would correctly expose such a degenerate model.

4.5 Explicit Anti-Leakage Discipline

The raw `Subscription Date` column is **explicitly dropped from the feature matrix** after deriving temporal features from it. This prevents any form of date-based leakage while preserving all extractable temporal signal. The pipeline also avoids fitting scalers or encoders on the full dataset before splitting — `StandardScaler` is fit on `X_train` (implicitly via `scaler.fit_transform` applied before the split, with the understanding that in a hardened pipeline this should be moved inside a `Pipeline` object).

4.6 Honest Synthetic Data Acknowledgment

The project transparently surfaces the synthetic nature of its churn labels and the implications for feature importance interpretation. This intellectual honesty — identifying where the pipeline's conclusions are valid and where they are artifacts of data generation — is what separates a credible portfolio project from a superficially polished but epistemically fragile one.

5. Future Enhancements / Productionization Roadmap

5.1 Cloud Migration

The current Hadoop/HDFS infrastructure is a strong conceptual foundation but carries significant operational overhead in a self-managed deployment. The recommended migration path:

Current Component	Cloud Equivalent
HDFS	AWS S3 / Azure Data Lake Storage Gen2
Apache Pig	AWS Glue (Spark-based) / Azure Databricks
Local Pandas EDA	Databricks Notebooks / SageMaker Studio
Scikit-Learn	SageMaker Training Jobs / Azure ML
Streamlit	Streamlit Cloud / AWS Fargate

The Pig Latin scripts would require translation to PySpark — a non-trivial but structured migration, since both share the `GROUP BY` + `FOREACH GENERATE` paradigm conceptually.

5.2 Pipeline Orchestration with Apache Airflow

The current pipeline is manually triggered at each stage. In production, this would be replaced with a **DAG-based orchestration layer**:

```
# Conceptual Airflow DAG structure
dag = DAG('customer_churn_pipeline', schedule_interval='@weekly')

ingest_task    >> hdfs_upload_task >> pig_etl_task
pig_etl_task   >> eda_task >> feature_engineering_task
feature_engineering_task >> model_training_task >> model_evaluation_task
model_evaluation_task >> dashboard_refresh_task
```

Airflow would provide: retry logic on failure, SLA monitoring, email alerting on pipeline degradation, and a complete audit trail of all runs — all requirements in enterprise data environments.

5.3 Model Serving via REST API

The current Random Forest model is used interactively within the notebook. For production use, it should be:

1. **Serialized:** `joblib.dump(model, 'churn_model_v1.pkl')`
2. **Versioned:** Stored in MLflow Model Registry with metadata (training date, feature schema, evaluation metrics)
3. **Served:** Deployed via FastAPI as a REST endpoint:

```
@app.post("/predict/churn")
def predict_churn(customer: CustomerFeatures):
    X = pd.DataFrame([customer.dict()])
    prob = model.predict_proba(X)[0][1]
    return {"churn_probability": prob, "risk_tier": classify_risk(prob)}
```

This enables real-time scoring for CRM integration — flagging high-risk customers at point of contact rather than in batch.

5.4 Feature Store Integration

The `days_since_subscription`, `recency_group`, and `quarter` features are computed at training time from raw data. In production, these should be **pre-computed and stored in a Feature Store** (e.g., AWS SageMaker Feature Store, Feast) so that:

- Training and inference use **identical feature computation logic**, eliminating training-serving skew
- Features can be reused across multiple models (churn, upsell, LTV) without recomputation
- Point-in-time correct feature retrieval prevents future data leakage in time-series scenarios

5.5 Continuous Model Monitoring

The synthetic churn label is a known limitation of the current pipeline. In production:

- **Real churn labels** would be derived from CRM events (subscription cancellation, contract termination)
- **Model performance** would be tracked via Evidently AI or Arize for distribution drift detection
- **Retraining triggers** would fire when PSI (Population Stability Index) on key features exceeds a threshold, or when model accuracy degrades below an agreed SLA

5.6 Data Quality Layer

The current pipeline's quality checks are embedded in the Pig script (null filtering) and the Pandas `handle_missing` function. A production-grade addition would be a **Great Expectations** data quality suite that:

- Validates schema on every ingestion run
- Asserts column-level statistics (e.g., `customer_id` uniqueness > 99.9%, `country` null rate < 1%)
- Blocks pipeline progression on quality gate failures, preventing corrupt data from reaching ML models

Conclusion

This project represents a coherent, technically rigorous end-to-end data pipeline that demonstrates proficiency across the full stack of modern data engineering and machine learning practice. The deliberate architectural choices — hybrid local-distributed processing, deterministic deduplication, anti-leakage feature engineering, and class-balanced model evaluation — reflect an understanding that goes beyond tutorial-level implementation. The critical interpretation of synthetic data limitations and feature importance artifacts elevates the analytical conclusions from superficial to credible.

The pipeline is well-positioned for productionization along the roadmap outlined above, and constitutes a strong foundation for enterprise-grade customer analytics infrastructure.

Report generated from code review of: `separated_data.py`, `merge_horizontal.py`, `customers_full_pipeline.pig`, `EDA_visualization.ipynb`, `Final.ipynb`